

A Comprehensive Exploration on Different Machine Learning Techniques for State of Charge Estimation of EV Battery

Mithul Raaj A T, Justin Ratnam, Niranjana Kumar S, Tanya Gupta, Keerthi Balaji, Rani C, Rajesh Kumar M, Mohamed Farrag

February 2026

State of Charge (SoC)

Definition

$$\text{SoC} = \frac{Q_{\text{remaining}}}{Q_{\text{max}}} \times 100\%$$

- $Q_{\text{remaining}}$ — remaining capacity
- Q_{max} — rated capacity

Different Methods for SoC Estimation

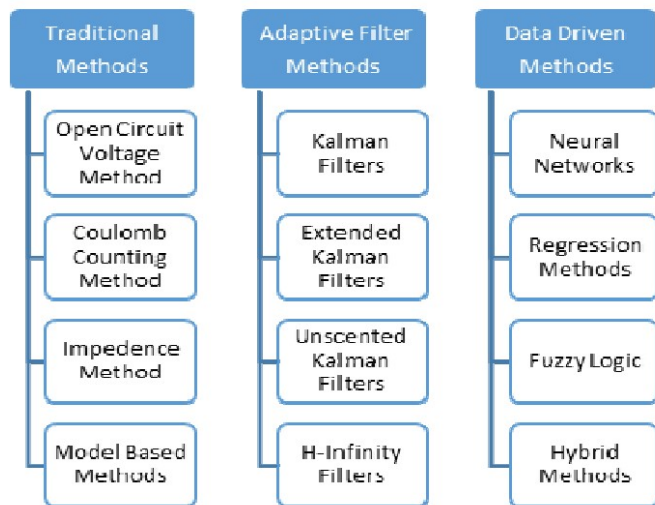


Рис.: Methods for State of Charge Estimation

Equivalent Circuit Model of Li-Ion Battery

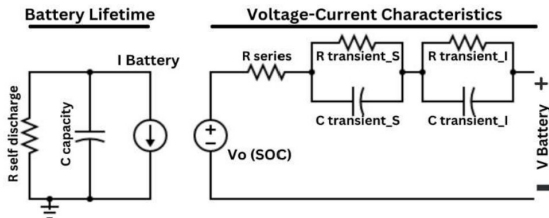


Рис.: Equivalent Circuit Model of Li-Ion Battery

Datasets Used in This Work

LG Dataset

- Experiments at temperatures: **0°C, 25°C, and 40°C**
- Real-world performance evaluation for EV applications
- Number of entries: **60,000–80,000**

UNIBO Dataset

- Data from **27 distinct battery cells**
- Multiple life stages — analyzes effect of cell aging on SoC
- Number of entries: **417,000**

Flowchart of the Proposed Methodology

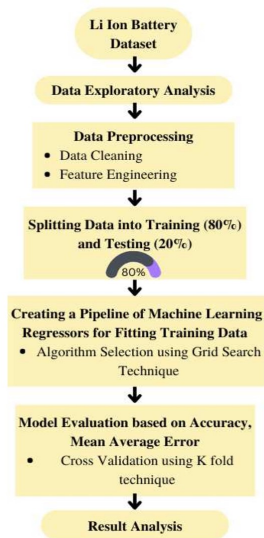


Рис.: Proposed Methodology Flowchart

Machine Learning Models Used in This Work

- Random Forest
- Decision Tree
- Gradient Boosting
- Support Vector Regression (SVR)
- Polynomial Regression
- Multilayer Perceptron (MLP)
- *Neural Networks*

Activation Functions

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Model Evaluation and Results

Evaluation Metrics

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Coefficient of Determination — R^2

R^2 Score

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- $R^2 = 1$ — perfect prediction
- $R^2 = 0.97$ — 97% of variance explained
- Scale-invariant, easy to compare across tests

Random Forest — How It Works

Основная идея

Random Forest — это ансамблевый метод, который строит множество **деревьев решений** и усредняет их предсказания.

Как работает

- 1 Создаётся N деревьев решений
- 2 Каждое дерево обучается на **случайной подвыборке** данных (bootstrap)
- 3 На каждом разбиении выбирается **случайное подмножество признаков**
- 4 Итоговое предсказание — **среднее** всех деревьев (для регрессии)

Преимущества

- Устойчив к переобучению
- Работает с нелинейными зависимостями

Decision Tree — How It Works

Основная идея

Decision Tree — это дерево решений, которое последовательно разбивает данные по значениям признаков.

Как работает

- 1 Выбирается признак и порог, которые лучше всего разделяют данные
- 2 Данные разбиваются на две группы
- 3 Процесс повторяется рекурсивно для каждой группы
- 4 Остановка при достижении максимальной глубины или минимального числа точек

Критерий разбиения (для регрессии)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

Gradient Boosting — How It Works

Основная идея

Gradient Boosting — это ансамблевый метод, который строит деревья **последовательно**, где каждое следующее дерево исправляет ошибки предыдущего.

Как работает

- 1 Начинаем с простой модели (например, среднее значение)
- 2 Для каждого следующего дерева:
 - Вычисляем **градиент** (ошибку) предыдущей модели
 - Обучаем новое дерево предсказывать этот градиент
 - Добавляем дерево к ансамблю с коэффициентом η (learning rate)
- 3 Повторяем M раз

Обновление модели

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Support Vector Regression — How It Works

Основная идея

SVR ищет гиперплоскость, которая предсказывает значения с ошибкой не более ϵ , максимизируя при этом "зазор" (margin).

Как работает

- Задаётся ϵ -трубка: ошибка внутри неё не штрафуются
- Точки вне трубки создают штраф, пропорциональный расстоянию
- Используется **ядровая функция** для учёта нелинейностей

Оптимизационная задача

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$$

при условии:

$$|y_i - (w \cdot x_i + b)| \leq \epsilon + \xi_i$$

Polynomial Regression — How It Works

Основная идея

Polynomial Regression — это расширение линейной регрессии, где добавляются **степенные признаки** для учёта нелинейностей.

Как работает

- 1 Исходные признаки преобразуются: $x \rightarrow [1, x, x^2, x^3, \dots, x^d]$
- 2 К полученным признакам применяется **линейная регрессия**
- 3 Модель остаётся линейной по коэффициентам, но нелинейной по исходным данным

Модель степени d

$$y = w_0 + w_1x + w_2x^2 + \dots + w_dx^d$$

Особенности

- Простая реализация

Multilayer Perceptron — How It Works

Основная идея

MLP — это полносвязная нейронная сеть с несколькими скрытыми слоями, которая обучается методом **обратного распространения ошибки**.

Как работает

- 1 Прямой проход (forward pass):

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = h(z^{(l)})$$

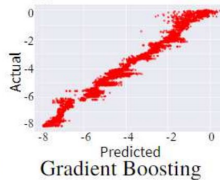
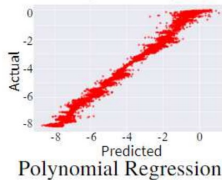
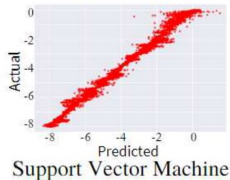
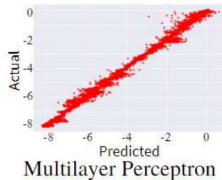
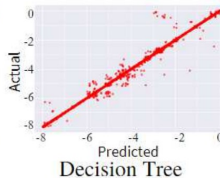
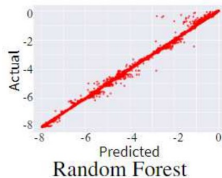
- 2 Обратное распространение (backpropagation):

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot h'(z^{(l)})$$

- 3 Обновление весов:

$$W^{(l)} := W^{(l)} - \eta \cdot \delta^{(l)} (a^{(l-1)})^T$$

Results Comparison



LG Dataset Results at 25°C

Таблица: LG Dataset Results at 0°C

Model	Accuracy (%)	MAE (%)
Random Forest	99.88	1.95
Decision Tree	99.76	1.28
Multilayer Perceptron	99.36	10.85
Support Vector Machine	99.13	14.16
Polynomial Regression	98.79	17.93
Gradient Boosting	98.59	18.72

Results Comparison

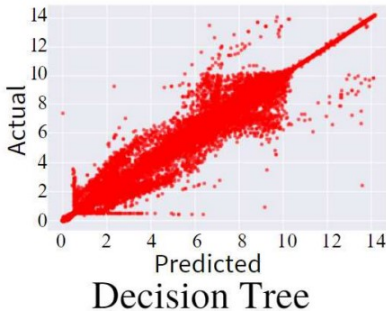
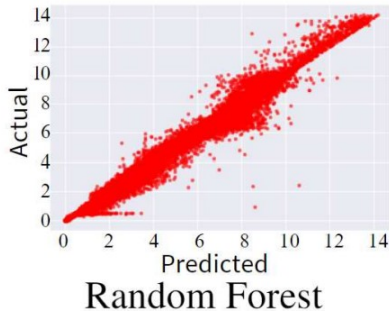


Рис.: Comparison of ML Models for SoC Estimation

UNIBO Dataset Results

Таблица: UNIBO Dataset Results

Model	Accuracy (%)	MAE (%)
Random Forest	99.16	13.26
Decision Tree	98.15	12.55

- **Gradient Boosting** and **Random Forest** show best performance across both datasets
- **MLP** achieves competitive results with $R^2 > 0.99$
- **LG Dataset** (60k–80k samples) — high accuracy at different temperatures
- **UNIBO Dataset** (417k samples) — validates model performance across cell aging
- Machine learning approaches effectively capture **non-linear battery behavior**
- Suitable for **real-time BMS applications** with low computational cost